Devups

Tome IV — Les modules fournis

Documentation du framework

Première édition — novembre 2026

Établie à partir du code réellement exécuté dans le projet **3agReader** (socle Devups 4.3).
Chaque affirmation porte sa source : *fichier:ligne*.

Sommaire

Chapitre 1 — Les composants du socle 5

1.1 La règle 5
1.2 devups — le noyau d'administration 5
1.3 devupscms — le contenu 5
1.4 devupsfront — le côté public 6
1.5 Les composants d'un projet réel 6

Chapitre 2 — ModuleTree : les données de référence 7

2.1 Le problème qu'il résout 7
2.2 Le modèle 7
2.3 branches — le schéma de profondeur 8
2.4 Les quatre modes de liaison 9
2.5 Le contrat de lecture 10
2.6 Les contraintes du modèle 10
2.7 chain est libre 11

Chapitre 3 — Le protocole de cache (C1 à C7) 12

3.1 Pourquoi un protocole 12
3.2 Ce qui est compilé 12
3.3 C1 — La clé de fichier 12
3.4 C2 — La langue 13
3.5 C3 — L'invalidation à l'écriture 13
3.6 C4 — L'atomicité 13
3.7 C5 — La lecture honnête 14
3.8 C6 — La profondeur 15
3.9 C7 — Ce qui n'est pas mis en cache 15
3.10 Le verdict de la seconde implémentation 16
3.11 Comment prouver qu'un cache est correct 16

Chapitre 4 — ModuleData : les statuts 17

4.1 Le principe : décrire, ne pas remplacer 17
4.2 Le modèle 17
4.3 La colonne value — décrire l'entier, pas le remplacer 17
4.4 L'API 18
4.5 Les appelants à vérifier avant de s'y fier 18
4.6 Un défaut d'écriture à connaître 19

Chapitre 5 — Les dettes connues du framework 20

5.1 Les dettes du cœur 20
5.2 Les leçons de méthode 21
5.3 Les points qui demandent un arbitrage de l'auteur du socle 21

Chapitre 6 — Récapitulatif pour qui reprend un projet 23

6.1 Les dix premières minutes 23
6.2 Les dix pièges qui coûtent le plus cher 23
6.3 Les commandes du quotidien 24
6.4 Comment contribuer à cette documentation 24

Ce tome suppose les trois précédents lus. Il décrit ce que le socle apporte déjà, pour éviter de le réécrire, et il consigne les dettes connues du framework.

Chapitre 1 — Les composants du socle

1.1 La règle

Trois composants arrivent avec le framework et doivent arriver **identiques** dans tout nouveau projet : `devups`, `devupscms`, `devupsfront`. Le composant métier porte le nom du projet.

> **Avant de coder un mécanisme, vérifier qu'un module du socle ne le fait pas déjà.** C'est la raison d'être de ce tome. Sur un projet réel, « j'ai besoin d'une table de statuts » et « j'ai besoin de catégories hiérarchiques » ont tous deux une réponse toute faite, et la recoder revient à maintenir deux fois le même concept.

1.2 `devups` — le noyau d'administration

Module	Rôle
`ModuleAdmin`	`Dvups_admin` (les comptes d'administration), `Dvups_role`, `Dvups_right` — les rôles et les droits du back-office
`ModuleConfig`	`Dvups_entity`, `Dvups_entity_lang`, `Configuration` — le registre de paramètres
`ModuleLang`	`Dvups_lang` — les langues ; c'est la table que lit toute la mécanique de traduction
`ModuleDatabase`	les outils de base de données

`Dvups_role` est l'endroit où `Core::updateDvupsTable()` écrit la liste des composants, modules et entités visibles (Tome I §7.5). Le rôle 1 est l'administrateur maître.

1.3 `devupscms` — le contenu

Module	Rôle
`ModuleTree`	**les données de référence en arbre** — voir chapitre 2
`ModuleData`	**les statuts génériques** (`Status`), les intervalles et barèmes (`Interval`, `Interval_group`) — voir chapitre 4
`ModuleCms`	`Cmstext` — des textes éditables, traduits
`ModuleGallery`	`Dv_image`, `Tree_item_image` — images et galeries
`ModuleLocation`	pays, villes, adresses
`ModuleNotification`	`Notification`, `Notificationtype`, `Notificationbroadcasted`
`ModulePushNotification`	notifications poussées (FCM)
`ModuleReporting`	`Reportingmodel` — tableaux de bord et rapports

1.4 `devupsfront` — le côté public

Module	Rôle
`ModuleUser`	`User`, l'authentification, les profils
`ModuleAnalytic`	`Analytic` — les mesures d'usage
`ModuleMessage`	`Conversation`, `Message` — la messagerie
`ModuleSlide`	`Slide` — les bannières

1.5 Les composants d'un projet réel

Sur 3agReader, en plus du socle :

Composant	Modules
`shopping`	`ModuleCatalog`, `ModuleOrder`, `ModulePayment`, `ModuleWallet`, `ModuleSticker`
`edition3ag`	`ModuleComicbook`, `ModulePost`, `ModuleContest`, …
`spacekola`	

Chapitre 2 — `ModuleTree` : les données de référence

2.1 Le problème qu'il résout

Dans toute application, il existe des **données meubles** : des valeurs qui remplissent des listes déroulantes et qui changent rarement. Des genres d'ouvrage. Des critères de notation. Des liens de réseaux sociaux. Des catégories.

Chacune, prise seule, appelle une table : un `id`, un libellé, parfois une description, parfois un parent, parfois une position. Toutes les mêmes colonnes, répétées.

`ModuleTree` les remplace par **une structure générique en arbre**, et compile chaque arbre en fichiers PHP pour un accès de type constante clé-valeur.

Le gain réel n'est pas d'économiser des tables : c'est de n'avoir **qu'un seul** écran d'administration, **un seul** mécanisme de traduction, **un seul** mécanisme de cache, pour toutes les énumérations du projet.

2.2 Le modèle

`tree` — un arbre

Colonne	Type	Note
`id`	int	
`name`	varchar(255)	**la clé machine** — unique, et **immuable en code**
`branches`	varchar(120) nullable	les noms de paliers, séparés par des virgules — **pilote la profondeur du cache**
`description`	text nullable	

`tree_item` — un élément

Colonne	Type	Note
`id`	int	
`tree_id`	FK `tree`	
`slug`	varchar(55) nullable	**la clé machine** — unique **par arbre**
`position`	int nullable	l'ordre d'affichage
`parent_id`	int nullable	auto-adjacence
`main`	int nullable	`1` = élément de premier niveau ; le filtre de `mainmenu()`

Colonne	Type	Note
`status`	int nullable	
`chain`	text nullable	**libre** — voir §2.7
`icon`, `image`, `uploaddir`		le média de l'élément

Plus `tree_lang` et `tree_item_lang` pour les libellés traduits.

La séparation déjà correcte

	Clé machine (non traduite)	Libellé (traduit)
`tree`	`tree.name`	`tree_lang.label`
`tree_item`	`tree_item.slug`	`tree_item_lang.name`

Le code se réfère toujours à la clé machine. Un libellé change quand un traducteur le change ; une clé machine ne doit jamais changer. C'est ce qui rend le cache clé-valeur possible.

2.3 `branches` — le schéma de profondeur

C'est le concept qui distingue `ModuleTree` d'une simple table auto-adjacente, et il mérite d'être compris avant de s'en servir.

`tree.branches` est la liste, séparée par des virgules, des **noms de paliers** de l'arbre, du plus haut au plus bas.

Une hiérarchie anonyme — « un élément, ses enfants, leurs enfants » — ne dit rien de ce que chaque niveau *est*. Nommer les niveaux leur donne une identité, et permet de s'y retrouver :

> Pour un arbre `classe` dont les éléments sont `6eme`, `5eme`, `4eme`…, on peut avoir une branche `série` au premier niveau (`A4`, `C`, `S`, `TI`…), puis une branche `filière` au deuxième, et ainsi de suite. L'analyse d'un projet permet de définir simplement ce modèle d'arborescence, et `ModuleTree` l'implémente alors directement.

Conséquences concrètes :

- **le gestionnaire d'arbre refuse de créer un sous-élément au-delà de la profondeur déclarée** — c'est `branches` qui fixe la limite, pas une constante du code ;
- **le cache descend sur `count(branches)` niveaux** (§3.6) ;
- chaque entrée du cache porte son `level` (1 = racine) et le nom de son palier (`branch`), calculés par le constructeur de cache lui-même, pas par un consommateur qui devrait reparcourir l'arbre ;
- un arbre **sans** `branches` garde un unique niveau.

Écriture. `Tree::setBranches($branches)` découpe sur la virgule, retire les espaces de bordure de chaque segment, et **refuse explicitement tout segment vide** (virgule en double ou en bordure) —

jamais un nettoyage muet. Conformément au contrat des setters (Tome II §5.4), il **retourne une chaîne d'erreur** plutôt que de lever.

Lecture. `Tree::getBranches()`. Mais pour un usage interne — la construction du cache — la lecture doit passer par une requête préparée brute :

```
$stmt = \Model::pdo()->prepare("SELECT branches FROM tree WHERE name = ?");
```

Raison : `Tree` est une entité traduite, et neuf des seize arbres réels de 3agReader n'ont aucune ligne `tree_lang`. Le défaut D15 (Tome II §7.4) les fait disparaître en silence du résultat de l'ORM — y compris l'arbre `menu`, le seul à avoir une hiérarchie réelle.

2.4 Les quatre modes de liaison

Une entité qui consomme une donnée de référence peut s'y rattacher de quatre façons. **Trois sont acceptables, une est proscrite.**

4.1 — Relation (une valeur)

```
/**
 * @ManyToOne(targetEntity="\Tree_item")
 * @JoinColumn(onDelete="set null")
 */
public $category;
```

La forme normale pour **une** valeur. Filtrable en SQL, intègre, jointe automatiquement.

Usages sur 3agReader : `ScanCore::$category`, `Dv_image::$folder`, `Dv_image::$position`, `Cmstext::$tree_item`, `Tree_item_image::$tree_item`, `Post::$category`.

4.2 — Table pivot (plusieurs valeurs)

Une table de liaison `<entité>_<référence>` avec deux clés étrangères.

La forme normale pour **plusieurs** valeurs. Filtrable, intègre.

Usages : `Comicbook_gender` (pivot `comicbook` ↔ `tree_item`), `Interval_group` (pivot `interval` ↔ `tree_item`, nommé `barem`).

4.3 — Instantané par slug (enregistrements historiques)

La valeur est figée dans la ligne au moment de sa création, par son `slug`, souvent dans une colonne JSON.

Réservé aux enregistrements historiques : une note de concours doit garder le critère sur lequel elle a été attribuée, même si le critère est renommé ou supprimé ensuite. Impose l'immuabilité du slug.

Usage : `Evaluation::criteria`, seul cas connu.

4.4 — Chaîne dénormalisée — proscrite

Les identifiants concaténés dans une colonne texte (`"3,7,12"`).

Pourquoi c'est proscrit : aucune intégrité, aucun filtrage SQL possible, et une divergence silencieuse dès qu'une référence est supprimée.

Un seul usage connu sur 3agReader : `comicbook.gender_names`, une colonne `varchar(55)` contenant des identifiants joints par des virgules — **migrée vers le pivot `comicbook_gender`** précisément parce qu'il fallait pouvoir filtrer par genre, ce que la chaîne rendait impossible.

> **Comment la dette se révèle.** Le commentaire laissé dans le client mobile après la migration dit tout : `category_id:eq` — confirmé en production ; `genders.id:eq` échoue (colonne inconnue côté SQL). Une liaison 4.4 n'est pas filtrable, et on ne s'en aperçoit que le jour où on en a besoin.

La règle

> Tout nouveau consommateur utilise **4.1** (une valeur) ou **4.2** (plusieurs valeurs). **4.3** est réservé aux enregistrements historiques et impose l'immuabilité du slug. **4.4** est proscrit.

Le mode n'est **pas déclaré** dans le datacore de l'entité consommatrice : il se lit dans son annotation. C'est une limite connue du format.

2.5 Le contrat de lecture

```
// la liste ordonnée d'un arbre, pour une liste déroulante
Tree::constant('criteria');

// un élément précis, par sa clé machine
Tree::constant('criteria', 'originalite');

// dans une langue donnée
Tree::constant('criteria', null, $id_lang);
```

`Tree::constant(string $name, ?string $slug = null, ?int $id_lang = null)` est le **point d'entrée unique** de lecture. `Tree::readCache($name)` lui délègue, conservé pour compatibilité.

2.6 Les contraintes du modèle

#	Contrainte	État sur 3agReader
U1	`tree.name` unique, non nul, `[a-z0-9_]+`	**posée** en base
U2	`tree.name` immuable une fois créé	**posée en code** : `Tree::__update()` compare à la valeur en base par requête préparée et lève sinon
U3	`tree_item.slug` unique **par arbre** (`(tree_id, slug)`)	**posée** ; l'index multi-colonnes autorise plusieurs `slug IS NULL` dans un même arbre, nécessaire pour U5

#	Contrainte	État sur 3agReader
U4	`tree_item.slug` immuable dès qu'un instantané 4.3 le référence	**non posée, volontairement** : la poser générique coupleraît `Tree_item` au seul module consommateur connu, sans protéger un futur consommateur dans un autre module
U5	`slug` non nul pour tout élément destiné au cache	**non conforme** : 12 éléments `main=1` avaient un slug nul, répartis sur 4 arbres ; corrigés par migration, mais rien n'empêche la situation de revenir
S1	assainissement du nom de fichier à l'écriture du cache	**posée** (`Util::arrayToPhpFile()`)

U2 est la contrainte qui fait fonctionner tout le reste : le cache est indexé par `tree.name`. Si le nom pouvait changer, le fichier de cache deviendrait orphelin sans que rien ne le signale.

2.7 `chain` est libre

La colonne `chain` servait à l'origine à stocker la liste des identifiants parents d'un élément, quel que soit son niveau. **Ce concept a été remplacé par `branches`**, qui est plus efficace et donne une identité à chaque niveau.

`chain` est donc désormais **polyvalente par conception** : chaque usage définit son propre format. Sur les arbres `configuration` et `network` de 3agReader, elle porte du texte libre sans rapport avec des identifiants.

Conséquence : il ne doit exister **aucun accesseur générique** qui présume un format. Celui qui existait (`getParentsCatTree()`) a été retiré après vérification qu'il n'avait aucun appelant réel — et pour une bonne raison : appelé sur un élément dont `chain` contient du texte libre avec une apostrophe, il levait une erreur SQL.

> **La leçon est transférable.** Une colonne volontairement polyvalente ne doit pas avoir d'accesseur typé. L'accesseur crée une promesse de format que la colonne ne tient pas.

Chapitre 3 — Le protocole de cache (C1 à C7)

3.1 Pourquoi un protocole

Compiler des données de référence en fichiers PHP est une idée simple qui va mal dans sept directions différentes. Les sept règles qui suivent ont été établies en construisant le cache de `ModuleTree`, puis **revérifiées une par une** en le réimplémentant pour `Status` — ce qui a montré lesquelles sont universelles et lesquelles dépendaient de `Tree`.

Ce protocole est la partie la plus réutilisable de ce tome. Il s'applique à tout cache de données de référence, dans n'importe quel projet.

3.2 Ce qui est compilé

`Tree::buildCacheData($treeName, $id_lang)` produit deux structures par langue active :

Fichier	Forme	Usage
`cache/trees/<name>.<iso>.php`	liste ordonnée	le rendu d'une liste déroulante, d'un menu
`cache/trees/<name>_references.<iso>.php`	carte indexée par slug	la recherche en temps constant

Le filtre : `main = 1`, `slug` non nul, triés par `position`, plus `count(branches)` niveaux d'enfants. Chaque entrée porte son `level` et son `branch`.

3.3 C1 — La clé de fichier

> **Le nom du fichier doit dériver d'une clé unique et immuable, et être assaini à l'écriture.**

Pour `Tree` : `tree.name`, unique (U1) et immuable (U2).

`Util::arrayToPhpFile()` assainit le nom par la regex `^[A-Za-z0-9_\-]+(\.[A-Za-z0-9_\-]+)*$`, qui autorise les séparateurs `<name>.<iso>` et `<name>_references.<iso>` mais bloque `../` et tout caractère hors de cette liste. `\InvalidArgumentException` sinon.

Adaptation mesurée pour `Status` : sa clé naturelle est `entity`, qui n'est **ni unique ni immuable** — c'est même le but qu'elle soit partagée, puisqu'une entité a plusieurs statuts. La clé de fichier est donc `entity` seul, et **un fichier contient toutes les lignes de cette entité**.

Et une conséquence imprévue, qui est exactement ce qu'un protocole doit rendre visible : la même regex **refuse un segment vide**, et les sept lignes `status` réelles ont `entity = ''`. Aucune ne peut donc avoir de fichier de cache tant qu'aucune entité réelle ne leur est assignée. C'est consigné en commentaire dans le code, pas découvert à l'exécution.

3.4 C2 — La langue

> **Un fichier par langue active.**

```
foreach (Dvups_lang::where("active", 1)->get() as $lang) { ... }
```

Inchangée pour `Status`.

> **Un écart corrigé, instructif.** La version antérieure du constructeur de cache de `Tree` ne transmettait pas `$id_lang` aux enfants : les éléments de premier niveau étaient dans la langue demandée, leurs enfants restaient dans la langue par défaut. Un cache multilingue doit propager la langue **à chaque niveau de la descente**.

3.5 C3 — L'invalidation à l'écriture

> **Toute écriture marque l'arbre comme sale ; la reconstruction est différée en fin de requête, depuis l'état final en base.**

```
Tree_item::__insert() / __update() / __delete()  →  Tree::markDirty($treeName)
Tree::__delete()                                  →  Tree::markDirty($treeName)
```

`markDirty()` enregistre **une seule** `register_shutdown_function` par processus, qui reconstruit en fin de requête chaque arbre marqué.

Trois propriétés qui comptent :

- **une reconstruction par arbre et par requête**, pas une par ligne écrite ;
- la reconstruction lit **l'état final** en base, jamais un état partiel au milieu d'une boucle d'écriture ;
- elle est **invisible dans le script qui écrit** : le rebuild différé ne peut être observé que dans un **second** processus PHP. Toute preuve d'invalidation doit donc se faire sur deux processus séparés — sinon on mesure l'absence de rebuild et on en conclut qu'il ne marche pas.

Inchangée pour `Status`.

3.6 C4 — L'atomicité

> **Fichier temporaire dans le même dossier, puis `rename()`. Exception si l'une des deux étapes échoue.**

```
$tmpPath = $targetPath . '.' . uniqid('tmp_', true);
file_put_contents($tmpPath, $phpContent);
if (!rename($tmpPath, $targetPath)) {
    @unlink($tmpPath);
    throw new \RuntimeException(...);
}
```

Sans cela, un lecteur concurrent peut inclure un fichier à moitié écrit — c'est-à-dire du PHP syntaxiquement invalide, donc une erreur fatale.

Inchangée pour `Status` (même fonction générique).

3.7 C5 — La lecture honnête

> **Un cache absent doit se voir, pas se taire.**

`Tree::constant()` :

1. lit le fichier de la langue demandée ;
2. si absent, **retombe sur la langue par défaut** ;
3. si absent aussi, **reconstruit depuis la base** ;
4. et dans ce cas : `error_log()` **toujours**, plus une `\RuntimeException` **si `!__prod`**.

```
$data = self::readTreeCacheFile($file, $iso);
if ($data === null && $iso !== $defaultIso) {
    $data = self::readTreeCacheFile($file, $defaultIso);
}
if ($data === null) {
    error_log($message);
    if (!__prod) throw new \RuntimeException($message);
    $built = self::buildCacheData($name, ...);
    $data = $slug === null ? $built['list'] : $built['references'];
}
return $slug === null ? $data : ($data[$slug] ?? null);
```

Deux détails qui font la différence :

- **`readTreeCacheFile()` renvoie `null`, pas `[]`**, quand le fichier est absent. Sans cette distinction, un cache manquant et un arbre réellement vide sont indiscernables, et C5 ne peut pas faire son travail.
- **`error_log()`, jamais `dv_dump()`.** `dv_dump()` tue la requête (Tome I §4.4) : signaler un cache absent ne doit pas faire tomber la page.

Inchangée pour `Status`, et sa sévérité apparente est le signal voulu. Comme aucune ligne `status` n'a d'`entity` non vide, `Status::constant('comicbook')` **lève aujourd'hui en développement** pour toute entité réelle. Ce n'est pas un bug du cache : c'est la preuve exacte du trou de données, rendue visible immédiatement au lieu d'être masquée par un `[]` silencieux.

3.8 C6 — La profondeur

> **La profondeur du cache suit le schéma déclaré, avec un garde anti-cycle et un plafond absolu.**

```
maxDepth = min(count(tree.branches), Tree::MAX_CACHE_DEPTH)   // MAX_CACHE_DEPTH = 10
```

Un arbre sans `branches` garde un unique niveau.

Deux garde-fous, et la façon dont ils ont été établis vaut d'être notée :

1. Cycle `parent_id`. Rien dans le schéma n'empêche `A → B → A`. Mesuré avant d'écrire le code : **zéro cycle** sur les 95 lignes réelles. `buildCacheNode()` trace néanmoins un ensemble `$visited` par élément racine, partagé par référence sur toute la descente ; un identifiant déjà vu arrête la descente **à ce nœud** (`error_log`, pas `dv_dump` qui tuerait la reconstruction de tout le cache pour un seul nœud corrompu) au lieu de boucler.

2. Plafond absolu `MAX_CACHE_DEPTH = 10`, indépendant de `count(branches)` : une valeur de `branches` démesurée ou corrompue ne peut pas faire descendre indéfiniment. 10 reprend une convention déjà présente dans le même fichier, très au-dessus de toute profondeur réelle (2 niveaux au maximum) ou plausible.

> **L'histoire de cette règle est le meilleur argument pour un protocole écrit.** La version antérieure disait « un seul niveau d'enfants ». Elle avait été écrite **arbitrairement**, et son auteur l'a lui-même signalé comme telle plutôt que de la laisser passer pour un choix réfléchi. Elle a été remplacée par une règle qui suit le schéma déclaré. Une règle de protocole non justifiée est une dette, pas une décision.

Non applicable à `Status` : les statuts sont plats, pas de `parent_id`, rien à adapter — pas une adaptation forcée.

3.9 C7 — Ce qui n'est pas mis en cache

> **Dire explicitement ce que le cache ne contient pas, et ce que cela coûte.**

Pour `Tree` : les éléments sans `slug` et ceux dont `main != 1` sont exclus (`whereNotNull('this.slug')`).

Conséquence réelle, consignée : 6 des 15 éléments de l'arbre `gender` n'avaient pas de slug — et **ne paraissaient donc jamais dans le cache des genres**, silencieusement, jusqu'à ce qu'un slug leur soit attribué.

C'est pour cela que C7 est une règle du protocole et pas une note de bas de page : **une exclusion silencieuse ressemble en tout point à une donnée absente.**

Adaptée pour `Status` : pas d'équivalent structurel, mais `entity = ''` joue le même rôle de fait — ces lignes ne sont jamais mises en cache par aucune entité, seulement lisibles via les accesseurs sans portée.

3.10 Le verdict de la seconde implémentation

Reprendre le protocole sur une deuxième entité est ce qui a montré sa solidité :

Règle	Verdict pour `Status`
C1 — clé de fichier	**adaptée** (clé non unique par nature)
C2 — langue	inchangée
C3 — invalidation	inchangée
C4 — atomicité	inchangée
C5 — lecture honnête	inchangée
C6 — profondeur	**non applicable** (données plates)
C7 — ce qui n'est pas mis en cache	**adaptée**

Quatre règles universelles, deux à adapter, une sans objet. **Un protocole qui n'aurait jamais été réimplémenté n'aurait pas su lesquelles.**

3.11 Comment prouver qu'un cache est correct

La méthode employée, reproductible :

1. **Équivalence élément par élément** entre la lecture par le cache et une lecture directe en base, sur **tous** les arbres réels — 16 sur 3agReader — y compris le cas limite d'un arbre à zéro élément.
2. **L'invalidation se vérifie sur deux processus PHP séparés** : le rebuild différé est invisible dans le script qui écrit (C3).
3. **Le cas limite « entité à zéro ligne »** se prouve directement, pas par simulation : un cache présent mais vide doit renvoyer `[]` **sans** exception, là où un cache absent doit lever (C5).
4. **Restaurer l'état d'origine** après la preuve. Sur 3agReader, `branches` a été placé temporairement sur l'arbre `menu`, le cache reconstruit et vérifié, puis `branches` remis à `NULL`.
5. **Dire ce qui n'a pas pu être prouvé en direct.** Le garde anti-cycle a été vérifié par une simulation isolée de l'algorithme, pas contre une vraie ligne en base : créer un cycle réel, même jetable, aurait exigé des lignes de traduction supplémentaires pour rester visible à l'ORM (D15), jugé disproportionné pour un garde-fou que les données réelles ne déclenchent pas.

Chapitre 4 — `ModuleData` : les statuts

4.1 Le principe : décrire, ne pas remplacer

Beaucoup d'entités ont une colonne `status` entière. `Status` est une table de **description** : une clé humaine (`draft`, `published`…) avec une couleur, une position, un libellé traduit, et une portée (`entity`).

Elle n'est jamais devenue la source de vérité de la colonne entière. Et c'est volontaire.

> **Pourquoi c'est la bonne séparation, pas un défaut à corriger.** `ScanCore` construit ses barrières de visibilité en SQL brut comparé à des entiers littéraux (`status = 1`), jamais par une jointure vers `status`. Faire dépendre une barrière de visibilité d'une table éditable par un administrateur — où l'on peut ajouter, renommer ou supprimer une clé — serait fragile, et hors du rôle d'un module d'affichage et de CRUD.
>
> **La table décrit ce que les entiers veulent dire pour un humain ; elle ne décide jamais ce qui est visible.**

4.2 Le modèle

Colonne	Rôle
`id`	
`_key`	la clé machine (`draft`, `published`…)
`value`	**l'entier décrit** (ajouté sur 3agReader)
`color`	la couleur d'affichage
`position`	l'ordre
`description`	
`entity`	la portée : à quelle entité ce statut s'applique

La contrainte `UNIQUE(entity, _key)` a été posée, pour empêcher deux lignes de porter la même clé pour la même entité — le défaut déjà vu ailleurs.

4.3 La colonne `value` — décrire l'entier, pas le remplacer

L'ajout d'une colonne `value` est la pièce qui réconcilie les deux mondes **sans** faire de la table la source de vérité : elle dit « la clé `draft` décrit l'entier 0 », et le code continue de comparer à 0.

Trois sources de vérité existaient sur 3agReader, et la méthode employée pour les réconcilier est reproductible :

1. **la table `status`** — descriptive ;

2. **`Chapter::getIconestatus()`** — un `switch` commenté, la source la plus explicite du code applicatif : 0 = brouillon, 1 = publié, 2 = en vérification, 3 = vérification validée, 4 = en attente de validation ;
3. **les commentaires SQL de visibilité de `ScanCore`** et `Comicbook::publish()` — une troisième source indépendante, qui ne corrobore que 0 et 1.

Clé	`value`	Confirmé par
`draft`	**0**	sources 2 et 3
`published`	**1**	sources 2 et 3
`in_pending`	**4**	source 2 (correspondance textuelle forte)
`process`, `end`, `suspended`, `succeed`	`NULL`	**non réconciliées**

> **Pourquoi quatre clés sont restées sans valeur, et pourquoi c'est la bonne décision.** Les entiers 2 et 3 existent dans le code, mais **aucune** des clés restantes ne porte un libellé qui leur corresponde textuellement. `process` (« En cours ») et `suspended` (« suspendu ») ne disent ni « vérification » ni « validé ». Et `end` et `succeed` portent **tous deux le même libellé anglais** (« Completed ») — ce qui est en soi la preuve que les libellés ne permettent pas de trancher de façon fiable.
>
> La distribution réelle des données, seule source qui aurait pu désambiguïser, ne le pouvait pas non plus : `comicbook.status` = {0: 64, 1: 48}, `chapter.status` = {0: 266, 1: 236}. **Aucune ligne réelle à 2, 3 ou 4**, bien que le code atteigne l'état 2.
>
> Deviner par proximité sémantique a été examiné puis rejeté. **Laisser `NULL` est une information ; deviner est une erreur silencieuse.**

4.4 L'API

```
Status::constant('comicbook');              // tous les statuts d'une entité
Status::constant('comicbook', 'draft');      // un statut précis
Status::constant('comicbook', null, $id_lang);

Status::get($key);                           // sans portée
Status::getStatus($key, $entityname);
Status::getStatusPosition($position, $entity);
Status::getStatusOf($entity);
$status->is($key);                           // test
$status->getColortab();                      // une pastille HTML colorée
$status->getValue();                         // l'entier décrit
```

`getColortab()` et `__toString()` produisent du HTML coloré. La couleur du texte est calculée par **luminance** (`textColorFor()`) au lieu d'être codée en dur — un noir sur fond sombre était illisible avant cette correction.

4.5 Les appelants à vérifier avant de s'y fier

Relevé sur 3agReader, non corrigé, signalé :

Appelant	Problème
`Subscription.php:141`	clé `"ongoing"` — **inexistante** parmi les clés réelles
`OrdersFrontController.php`	clé `"confirm"` — également inexistante
`PostTable.php`	`Post::where(Status::getStatus("published"))` — même défaut structurel qu'un `Comicbook::active()` corrigé depuis
`Model.php:811`	`Status::getStatus($key, strtolower($classname))` — générique, dans `dclass/`

Une clé inexistante ne lève pas : elle renvoie une entité vide (Tome II §4.5). Le filtre devient alors silencieusement faux.

4.6 Un défaut d'écriture à connaître

```
// NE FONCTIONNE PAS : SQL malformé
(new Status($id))->__update();

// FONCTIONNE
Status::where('this._key', '=', $key)->first()->__update();
```

`new Status($id)` suivi de `__update()` produit un SQL malformé (racine dans `Model.php` et `DBAL.php`). Hydrater par le chemin standard avant de mettre à jour. C'est la même famille de problème que le `dvold` du Tome II §2.4.

Chapitre 5 — Les dettes connues du framework

Ce chapitre existe pour une raison précise : **un défaut connu et nommé coûte une heure ; le même défaut redécouvert coûte une journée.** Chacun porte son état réel.

5.1 Les dettes du cœur

#	Dette	Emplacement	État
D1	**D15** — `INNER JOIN dvups_lang` chaîné après un `LEFT JOIN <entité>_lang` : une ligne traduite sans traduction disparaît de **toute** lecture ORM	`QueryBuilder::initSelect()`	**non corrigé** — racine partagée par toute entité traduite, dans tout projet. Contournement : forcer la langue
D2	colonne **ambiguë** sur une entité traduite : `l.name` de `dvups_lang` entre en collision avec la colonne `name` de l'entité	`QueryBuilder::initSelect()`	**corrigé côté appelants seulement** (7 occurrences qualifiées) ; la racine reste
D3	`$dvold` posé par **référence** dans `hydrate()` : l'`UPDATE` n'écrit rien	`Model::hydrate()`	**contourné dans le code généré** (`find()` au lieu de `new X($id)`) ; racine non corrigée **par décision**
D4	`get($col, $id_lang)` : le second argument est **sans effet** sur la langue	`QueryBuilder::get()`	**non corrigé** ; utiliser `setLang()` avant
D5	`whereIn()` quote sans échapper (`qb_sanitize`)	`QueryBuilder`, `dvups_functions.php`	**non corrigé** — valider avant
D6	`whereBetween()` interpole directement	`QueryBuilder`	**non corrigé**
D7	un opérateur hors de `$operators` **ignore la valeur** et lie l'opérateur comme paramètre	`QueryBuilder::where()`	comportement du framework
D8	`whereRaw()` et `where_str()` ont un corps **identique**	`QueryBuilder`	laissé tel quel : renommer casserait des appelants pour un gain cosmétique
D9	`new Status($id)` puis `__update()` : SQL malformé	`Model.php` + `DBAL.php`	**non corrigé**
D10	montants en `type="float"` (`totalamount`, `commission`, `discountprice`, `nettopay`, `kolacoin`)	`dclass/extends/OrderCore.php`	**non corrigé** — arithmétique d'argent non exacte
D11	`nullable` d'une **relation** ignoré par le générateur	`BackendGenerator.php`	**non corrigé** — la colonne de clé étrangère est NULLable dans tous les cas
D12	`core:g:crud` **duplique** le bloc de `require` du module	`BackendGenerator.php`	**non corrigé** — relire le fichier de module après régénération
D13	`schema:update --force` répond « success » sans appliquer un renommage d'index	Doctrine / `builder.php`	contournement : appliquer le SQL de `--dump-sql` directement
D14	`install` : `config_data.sql` commence par ses propres `CREATE TABLE` et échoue en silence	`builder.php` + `config_data.sql`	**non corrigé** — rattrapage manuel (Tome I §3.2)

#	Dette	Emplacement	État
D15	`Router::cacheRoute()` commence par `return null` : le cache de routes n'est **jamais écrit**	`Router.php:697`	mécanisme complet en lecture, mort en écriture
D16	le contrôle de session de `manageServe()` est **commenté**	`Router.php:546-550`	l'API du back-office ne vérifie pas la session en tête de porte
D17	`FORMTYPE_INJECT_COLLECTIONFORM` déclaré, **aucune branche de rendu**	`FormManager.php` + `FormFactory.php`	à ne pas utiliser
D18	`Datatable` : une clé de colonne qui ne nomme aucune propriété réelle **casse l'écran entier** (via le `die()` de `ModelTrait::__get()`)	`Datatable.php` + `ModelTrait.php`	comportement du framework — la clé doit nommer une colonne réelle
D19	`Dvups_config_item` déclare `dvtranslate = true`, n'a **aucune table** et **aucun appelant**	`dclass/extends/`	code mort

5.2 Les leçons de méthode

Celles-ci ne sont pas des défauts mais des règles de travail, chacune payée par une erreur réelle.

1. Mesurer avant de corriger. Plusieurs corrections ont visé des défauts qui n'existaient plus, ou qui portaient sur du code mort. Avant de corriger, vérifier que le chemin est **vivant** : une méthode publique et bien nommée peut être inatteignable (Tome III §2.2), un appel peut être précédé d'un `return ""`, un bloc peut être dans un commentaire ou derrière un `@if(false)`.

2. Relire la source, pas une copie. Une mesure faite sur une copie d'un fichier prise plus tôt dans la session peut contredire la réalité. Relire juste avant de conclure.

3. Une règle de protocole non justifiée est une dette. La règle C6 avait été écrite arbitrairement. Son auteur l'a signalée comme telle plutôt que de la laisser passer pour un choix. Elle a été remplacée.

4. Un `die` manque, et un refus de sécurité n'en est plus un. Deux occurrences réelles dans `systemServe()` : le refus était bien émis, et l'écriture se faisait quand même. **Après tout `json_encode()` de refus, vérifier qu'un `die` suit.**

5. Nommer ce qu'on n'a pas pu prouver. « Vérifié par simulation isolée, pas contre une vraie ligne en base » est une information utile. « Vérifié » tout court, quand ce n'est pas le cas, est une dette cachée.

6. Ne pas deviner une donnée manquante. Les quatre clés de statut sans valeur, les douze slugs absents, les sept lignes à `entity` vide : tous **nommés et laissés au propriétaire**, pas remplis par proximité sémantique.

7. Qualifier les colonnes. `this.<colonne>`, toujours. Cela ne coûte rien et prévient le jour où l'entité devient traduite.

5.3 Les points qui demandent un arbitrage de l'auteur du socle

Ces quatre décisions dépassent un projet et ne devraient pas être prises dans un chantier applicatif :

Point	Enjeu
la jointure D15	la corriger change le résultat de chaque lecture traduite, dans tout projet basé sur ce socle
`$dvold` par référence	`clone $this` marche, c'est testé — mais touche le cœur du framework ; le correctif vit aujourd'hui dans le gabarit généré, donc il est à réappliquer après chaque régénération
les montants en `float`	une migration de type sur des colonnes d'argent, dans `dclass/extends/`
le gabarit de `update()`	tant qu'il n'est pas corrigé, toute régénération réintroduit le défaut D3

Chapitre 6 — Récapitulatif pour qui reprend un projet

6.1 Les dix premières minutes

Faire	Pourquoi
ouvrir `config/constante.php`	la base, les URLs, `__prod`, les secrets
ouvrir `src/requires.php`	la liste réelle des modules chargés
ouvrir `route/WebService.php`	`$public_access` et `$publicMethods` : la surface d'API et son modèle de sécurité
lancer `php devups schema:update --dump-sql`	si ce n'est pas vide, la base et le code divergent
lister `src/`	les composants ; ceux en `devups*` sont le socle
chercher les fichiers `*Core.json`	tout dossier sans le sien est invisible
ouvrir `aiMemory/` ou l'équivalent, s'il existe	la mémoire du projet

6.2 Les dix pièges qui coûtent le plus cher

1. **Un module sans son `*Core.json` est invisible**, sans erreur (Tome I §7.3).
2. **`find()` ne renvoie jamais `null`** : tester `if ($x && $x->getId())` (Tome II §4.5).
3. **Une relation jointe ne porte que l'`id`** : `with()` est requis (Tome II §4.4).
4. **`new X($id)` puis `__update()` n'écrit rien** : utiliser `find($id)` (Tome II §2.4).
5. **Une entité traduite sans traduction est invisible à l'ORM** (Tome II §7.4).
6. **`gte` / `lte` n'existent pas** dans les filtres : un opérateur inconnu devient un `LIKE` (Tome III §4.7).
7. **`lazyloading` est paginé à 30, silencieusement** ; `?next=` et `?per_page=`, pas `?page=` (Tome III §2.1).
8. **`?relation_id=X` ne filtre rien** ; `?dfilters=on&relation.id:eq=X` (Tome III §2.1).
9. **Une méthode de `FrontController` sans annotation de verbe est morte** (Tome III §2.2).
10. **Régénérer une entité écrase les contraintes ajoutées à la main** et duplique ses `require` (Tome I §6.7).

6.3 Les commandes du quotidien

```
php devups serve                              # le serveur de développement
php devups schema:update --dump-sql           # lire avant d'appliquer
php devups schema:update --force              # appliquer
php devups dvups_:update                      # après toute création de module ou d'entité
php devups core:g:crud <Comp>\<Module>\<entité>
php devups core:g:frontcontroller <Comp>\<Module>\<entité>
composer dump-autoload                         # si une classe « n'existe pas » alors qu'elle
est là
```

6.4 Comment contribuer à cette documentation

Ce document est destiné à être itéré, projet après projet. Trois règles pour qu'il reste fiable :

1. Toute affirmation porte sa source. `fichier:ligne`, ou « mesuré le en ». Une affirmation sans source est une supposition qui a l'air d'un fait.

2. Signaler les divergences entre versions du socle plutôt que de les écraser. Ce document contient déjà un cas : le mémo de BGFIBank affirme qu'une route de sous-ressource par annotation n'est jamais lue, et le code de 3agReader la lit bien. **Les deux affirmations sont vraies, sur deux versions différentes.** Écraser l'ancienne aurait perdu l'information que le comportement a changé.

3. Nommer ce qui n'est pas couvert. L'avant-propos du Tome I liste les zones non documentées. Cette liste est aussi utile que le reste : elle dit où ne pas chercher une réponse qui n'y est pas.

Fin du Tome IV, et de cette première édition.